

which response is better. As shown in Figure 3, our method consistently outperforms all baselines on both medical and financial QA tasks. More implementation details are provided in Appendix E. We also provide a case study in Figure 7 and Appendix F to further demonstrate the effectiveness of our method.

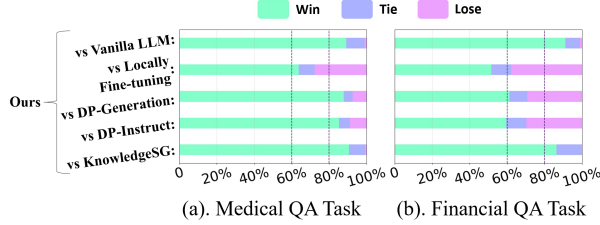


Figure 3: Using LLM-Judge to compare the outputs generated by our method with those of other baselines. **Win** means our method outperformed the baselines, **Tie** means the results were similar, and **Lose** means our method performed worse than the baselines.

6.4 Empirical Privacy Protection Results

In this section, we implement Data Extraction Attack (Carlini et al., 2021) and Membership Inference Attack (Yeom et al., 2018; Choquette-Choo et al., 2021) on our method and baselines to empirically evaluate the privacy protection capability. For those baselines, including DP-Generation/DP-Instruct/KnowledgeSG, the client only transfers the generation proxy model to the server. In contrast, *RewardDS* transfers both the generation proxy model and the reward proxy model. Accordingly, the attack targets for the baselines are limited to the generation proxy model, while for *RewardDS*, both models can be attacked. We also provide the attack results of No Protection, which serves as the upper bound.

As shown in Figure 4, *RewardDS* demonstrates superior privacy protection capacity compared to those baselines, as indicated by its comparable ROUGE-L scores and significantly lower F1 scores. This is possibly due to *RewardDS* allocating the privacy budget for both generation proxy model and reward proxy model, thereby reducing the privacy budget of each individual model and making them more difficult to be attacked.

Implementation details of these attack methods can be found in Appendix G.

6.5 In-depth Analysis of *RewardDS* Design

Here, we provide more detailed analysis on the design and effectiveness of *RewardDS*.

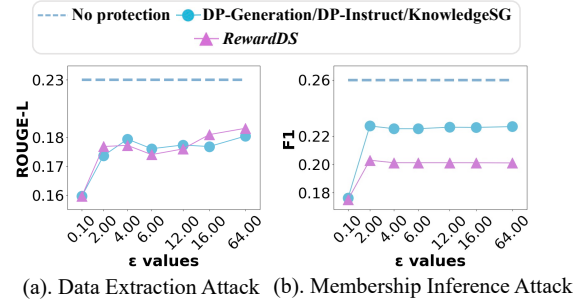


Figure 4: Results of Data Extraction Attack and Membership Inference Attack for *RewardDS* and all baselines under different privacy budgets ϵ on Medical QA task.

Analysis 1: Impact of *RewardDS* on Synthetic Data Quality and Downstream Performance.

According to Alg. 1, *RewardDS* iteratively refines the synthetic data during each training epoch. As shown in Figure 5(a), the reward score of synthetic data gradually increases with iterative refinement, indicating improved data quality.

We also evaluate the downstream performance of the target LLM on the Medical QA task after being fine-tuned on the synthetic data from different refinement stages. The results in Figure 5(b) show that the downstream performance of the fine-tuned LLM also improves significantly with the improvement of the synthetic data quality, strongly highlighting the effectiveness of *RewardDS*.

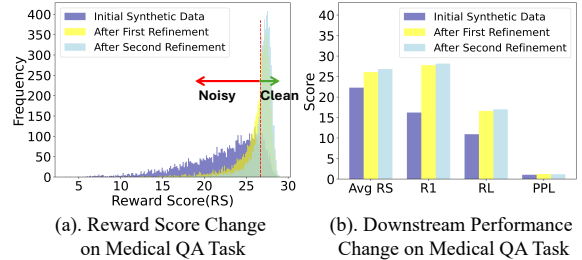


Figure 5: Changes of Reward Score and Downstream Performance in *RewardDS* for the Medical QA Task. **Avg RS** indicates the average reward score of the synthetic data, while **R1**, **RL**, and **PPL** represent the downstream performance metrics of the target LLM fine-tuned using these synthetic data.

Analysis 2: Training Cost Analysis of *RewardDS*.

As shown in Figure 2, *RewardDS* introduces additional modules, including Reward Proxy Model Training, Reward-Guided Filtering, and Self-Optimizing Refinement, into the privacy-preserving fine-tuning process of the server-side LLM. We measure the additional time cost of these modules and compare it with that of the original modules: Generation Proxy Model Training and

LLM fine-tuning. As shown in Table 2, the additional time cost from our method accounts for **only 29.69%** of the total time cost, with most of the time consumed by LLM fine-tuning modules. This is primarily due to the use of a lightweight reward proxy model, making the associated modules highly efficient. Overall, the additional time cost introduced by *RewardDS* is minimal, strongly demonstrating the practicality of our method.

	Time (min)	Percentage
Initial Modules:	315	70.31%
Generation Proxy Model Training	45	10.04%
LLM fine-tuning	270	60.26%
Additional Modules From <i>RewardDS</i>:	133	29.69%
Reward Proxy Model Training	49	10.94%
Reward Guided Filtering	12	2.67%
Self-Optimizing Refinement	72	16.07%

Table 2: Runtime of different modules in *RewardDS* for privacy-preserving fine-tuning on Medical QA task. The most time-consuming module is marked in **bold**.

Moreover, we compare the overall training time and GPU memory usage of *RewardDS* with other baselines in Table 3. As for training time cost, our method incurs only a small overhead, primarily from dynamically filtering and refining synthetic data to enhance the overall quality of these data. For the GPU memory cost, we only introduce an additional lightweight reward proxy model with 0.5B parameters, which is relatively small and highly efficient. Overall, with just a slight extra cost in training time and GPU memory, our method can achieve significant performance improvements, as shown in Table 1, making it highly practical for real world deployment.

Method	Time Cost (min)	Total Parameter (B)
DP-Generation	315	7 + 0.5
DP-Instruct	320	7 + 0.5
KnowledgeSG	166	7 + 0.5
<i>RewardDS</i>	448	7 + 0.5 + 0.5

Table 3: Comparison of the time cost and the GPU memory cost between our method and the other baseline methods. The GPU memory cost are measured by the total model parameter amount.

Analysis 3: *RewardDS* vs Filtering according to Reward Score.

According to Figure 5(a), the initial synthetic data contains substantial samples with low reward scores. One straightforward strategy is to filter out these low-quality samples to reduce noise. As

shown in Table 4, simply selecting Top 50% synthetic data for fine-tuning can improve the overall data quality and slightly enhance downstream performance. However, filtering also reduces the size of the training set. As more data is discarded, the downstream performance begins to drop due to the limited training data.

In contrast, *RewardDS* applies Self-Optimizing Refinement to improve the quality of low reward samples instead of discarding them. **It can significantly enhance data quality while maintaining a stable dataset size.** Consequently, *RewardDS* achieves superior downstream performance, as demonstrated in Table 4.

	Data Count $\uparrow$	Avg RS $\uparrow$	Downstream RL $\uparrow$
Raw Synthetic Data	6420	22.30	10.94
– Select Top 80%	<u>5136</u>	24.05	11.09
– Select Top 50%	3210	25.66	<u>12.43</u>
– Select Top 30%	1926	26.53	11.19
– Select Top 10%	642	27.43	6.82
Refinement by <i>RewardDS</i>	6420	<u>26.82</u>	17.02

Table 4: Comparison between filtering synthetic data only based on Reward Score (RS) and iterative refinement using *RewardDS* on Medical QA task. Higher Avg RS indicates better overall data quality. The best results are shown in **bold**, and the second-best results are underlined.

Other Analysis: Furthermore, we analyze the impact of different privacy budget allocations for our method in Appendix H. Those results in Figure 8 also clearly demonstrate the effectiveness and robustness of our method.

6.6 Hyperparameter Analysis

In this section, we conduct hyperparameter analysis to further prove the effectiveness of our method.

Privacy Budget ϵ . We evaluate the performance of our method and baseline approaches under varying privacy budgets ϵ . As shown in Figure 6, our method consistently outperforms all baselines, not only under the commonly used setting of $\epsilon = 16$, but also under stricter privacy budgets (e.g., $\epsilon = 2, 4$, and 6). These results demonstrate the superior performance and broad applicability of our method.

Synthetic Data Count L . To further validate the effectiveness of our method, we compare its performance with baseline methods under different synthetic data count L . As shown in Table 5, *RewardDS* consistently outperforms all baselines across different data counts, demonstrating its robustness and effectiveness. Notably, even when the

Synthetic Data Count	642(1%)	1926(30%)	3210(50%)	5136(80%)	6420(100%)
DP-Generation (Kurakin et al., 2023)	6.71	9.55	10.60	10.30	10.94
DP-Instruct (Yu et al., 2024)	6.63	8.00	7.99	8.45	8.44
KnowledgeSG (Wang et al., 2024)	9.43	10.21	10.54	10.92	10.74
RewardDS	16.46	17.17	17.39	17.34	17.02

Table 5: Performance comparison between *RewardDS* and the other baseline methods under different synthetic data count for the Medical QA task.

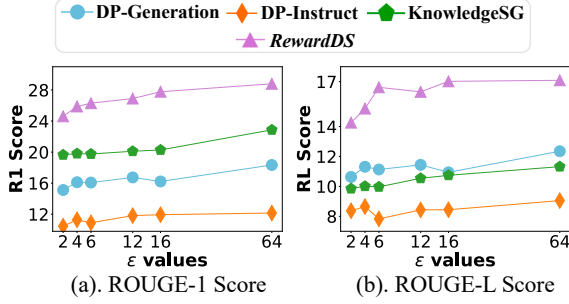


Figure 6: Performance of *RewardDS* and baselines on the Medical QA task under different privacy budgets ϵ for the Medical QA task. The performance is evaluated using ROUGE-1 (**R1**) and ROUGE-L (**RL**) scores.

synthetic data—for example is extremely scarce, only 1% (642 samples), our method still maintains competitive performance while other baselines suffer from performance degradation. This superiority is mainly attributed to our self-optimizing refinement module, which effectively improves the quality of those limited synthetic data and can maximize their utility in contrast to the baselines.

More Hyperparameters. We also analyze more hyperparameters in our method described in Alg. 1, including the number of folds k and the number of candidate responses N . As shown in Figure 9, 10 and 11, our method remains effective and robust across different hyperparameter settings on three domain-specific tasks. More detailed analyses can be found in Appendix I.

6.7 Generalization to More Proxy Models and LLMs

We also evaluate the performance of our method when using different combinations of generation proxy models and reward proxy models, as shown in Table 6. To explore the impact of proxy model heterogeneity, we evaluate the combination of Qwen-0.5B + LLaMA3-1B. To explore the impact of proxy model scale, we consider the combination of Qwen-0.5B + Qwen-1.5B. As shown in the table, using Qwen-0.5B as the generation proxy

model and Qwen-1.5B as the reward proxy model achieves the best performance. This is mainly because both proxy models share a similar architecture, which ensures more effective collaboration between them. Moreover, a larger reward proxy model can more effectively enhance the quality of synthetic data and achieve better performance, which underscores the critical role of the reward proxy model.

Generation Proxy	Reward Proxy	RL Score	R1 Score	PPL
Qwen-0.5B	Qwen-0.5B	17.02	27.78	1.17
Qwen-0.5B	Llama3-1B	17.69	28.71	1.21
Qwen-0.5B	Qwen-1.5B	18.11	30.30	1.18
Llama3-1B	Qwen-0.5B	12.52	19.35	1.15
Qwen-1.5B	Qwen-0.5B	17.44	29.64	1.24

Table 6: Performance of our method under different combinations of generation proxy models and reward proxy models for the medical QA task.

Moreover, we evaluate the performance of our method with different server-side LLMs, such as Llama-2-7B-chat-hf (MetaAI, 2023) and Qwen2.5-14B-Instruct (Yang et al., 2024b) in Appendix H. As shown in Table 8, our method consistently outperforms other baselines across various LLMs, demonstrating its effectiveness and robustness.

7 Conclusion

We propose a novel privacy-preserving framework, *RewardDS*, to mitigate noise in synthetic data during LLM privacy-preserving fine-tuning. Specifically, *RewardDS* fine-tunes a reward model and leverages the reward signal to guide the synthetic data generation process. During the data synthesis process, *RewardDS* employs the collaboration of Reward Guided Filtering and Self-Optimizing Refinement modules to filter and refine synthetic data, mitigating noise. We conduct extensive experiments across medical QA, legal QA, and code generation tasks. The results consistently demonstrate the effectiveness of *RewardDS* for privacy-preserving LLM fine-tuning.